

Performance Analysis: OpenMP MergeSort vs. CUDA Bitonic Sort

High-Performance Computing

Abdulrahman Osama Adel (B.N.: 13)
Mahmoud Atef Mahmoud AbdelAziz (B.N.: 33)
Mohamed Alaa Ahmed Refaee (B.N.: 30)

March 25, 2026

1 Introduction

Sorting large datasets is a fundamental problem in computer science. This project evaluates two distinct parallelization strategies: a divide-and-conquer approach using OpenMP (MergeSort) and a sorting network approach using CUDA (Bitonic Sort). The objective is to analyze compute times, measure parallel speedup against a standard serial baseline, and evaluate how different hardware architectures handle data-dependent versus data-independent branching.

2 Algorithm Design and Implementation Details

2.1 Serial Baseline

To ensure a strict, apples-to-apples comparison with the OpenMP implementation, the serial baseline was implemented as a custom, recursive Divide-and-Conquer MergeSort rather than utilizing the C++ standard library's `std::sort` (which relies on Introsort). The function `serial_sort()` guarantees a consistent worst-case and average-case time complexity of $O(N \log N)$.

A critical optimization in this serial baseline is its memory management strategy. Instead of dynamically allocating temporary arrays at each level of the recursion tree, a single auxiliary `scratch` vector of size N is allocated exactly once in the main wrapper function. This buffer is passed by reference to the recursive `serial_merge_sort_recursive()` helper. The recursion splits the array, recursively sorts the halves, and utilizes `std::merge` to combine the ordered segments into the scratch buffer before copying them back to the original array. This approach completely eliminates the severe performance penalty of dynamic memory allocation during deep recursion, providing a highly efficient and fair ground-truth baseline for both correctness verification and speedup calculations.

2.2 OpenMP MergeSort

The OpenMP implementation follows the classical divide-and-conquer structure of MergeSort. The recursive routine `merge_sort_task()` splits the range $[lo, hi)$ into two halves, recursively sorts each half, and then combines them using `std::merge`. To avoid repeated temporary allocations during recursion, the implementation allocates a single auxiliary vector `scratch` once in `omp_merge_sort()` and passes it through all recursive calls. The actual merge is performed in two phases: the ordered halves are first merged into the corresponding segment of `scratch`, and then copied back into the original array. This preserves the expected MergeSort work complexity of $O(N \log N)$ while keeping the implementation straightforward and deterministic.

Parallelism is introduced with OpenMP tasking. Inside a single OpenMP parallel region, the root thread enters the recursion through `#pragma omp single nowait`, and each sufficiently large subproblem may create two child tasks using `#pragma omp task`. A `#pragma omp taskwait` then enforces the required synchronization before merging, ensuring that both halves are completely sorted before the combine step begins. This task-based design is well matched to recursive MergeSort because different branches of the recursion tree can execute independently and be scheduled dynamically across threads.

A crucial design choice in this implementation is the adaptive serial cutoff, controlled by the parameter `cutoff`. When the subarray length satisfies `len ≤ serial_cutoff`, the code does *not* continue recursing in parallel; instead, it falls back to `std::sort` on that subrange. By switching to `std::sort` for small ranges, the implementation leverages a highly optimized serial kernel exactly where fine-grained parallelism would be least effective.

The code further reinforces this idea by only enabling actual task creation when `len ≥ 2 × serial_cutoff`, through the boolean `spawn_tasks`. For problem sizes only slightly above the cutoff, the recursion continues without generating deferred tasks, which prevents over-decomposition near the leaves of the recursion tree. This makes the OpenMP design explicitly hybrid: coarse-grained subproblems are handled through task parallelism, while fine-grained subproblems are delegated to the optimized serial library sort. Such a structure is typical in high-performance recursive algorithms because it balances available parallelism against runtime overhead.

2.3 CUDA Bitonic Sort

The GPU implementation uses Bitonic Sort, a sorting-network algorithm particularly well suited to SIMD/SIMT architectures such as CUDA GPUs. Unlike comparison sorts whose control flow depends on the data, a bitonic sorting network has a fixed sequence of compare-exchange operations determined entirely by index arithmetic. In the implementation, this is visible in the nested progression over the network stages `k` and `j`, and in the partner computation `partner = idx ^ j`. The direction of comparison is likewise computed from the index pattern rather than from the input values. This data-independent structure is attractive on GPUs because it yields regular memory access and predictable control flow, although its asymptotic complexity is $O(N \log^2 N)$, which is theoretically less efficient than $O(N \log N)$ methods such as MergeSort.

A first important design choice is the padding strategy for arbitrary input sizes. Bitonic Sort requires the logical problem size to be a power of two, so the host code computes the next power of two using `next_power_of_two()`. It then allocates a host-side buffer of that padded length and initializes the extra entries to `INT_MAX`. The original

data are copied into the prefix of this buffer, transferred to the device, sorted, and finally only the first `original_size` elements are copied back into the user-visible array. Padding with `INT_MAX` is a particularly clean choice for ascending order because these sentinel values naturally migrate to the end of the sorted sequence and therefore do not disturb the ordering of valid inputs. This allows the implementation to support arbitrary array lengths while preserving the correctness requirements of the bitonic network.

A second major design choice is the explicit use of shared memory to accelerate the early stages of the sorting network. The kernel `shared_bitonic_tile_kernel` assigns each block a tile of $2 \times \text{block_size}$ elements, loads that tile into `__shared__` memory, performs the complete local bitonic schedule there, and writes the results back to global memory. Because shared memory has substantially lower latency than global memory, this optimization reduces the cost of the dense compare-exchange operations that occur in the initial stages, where all compare partners remain within the same tile. The kernel also pads out-of-range tile entries with `INT_MAX`, which preserves correctness at the boundaries of the padded array.

After these local stages are completed, the implementation transitions to the global-memory kernel `bitonic_step_kernel`. This occurs once the bitonic sequence length exceeds the tile size, i.e., when compare-exchange partners may lie in different thread blocks and can no longer be handled entirely within shared memory. At point, the algorithm launches successive global kernels for the remaining `k` and `j` stages. Each thread handles one candidate index, computes its XOR partner, and only one thread per pair performs the exchange by rejecting cases where `partner <= idx`. This avoids duplicate swaps while preserving the exact structure of the sorting network.

Overall, the CUDA design is therefore a two-level implementation of Bitonic Sort: a shared-memory phase for fast intra-tile ordering, followed by a global-memory phase for larger cross-tile merges. This hybridization reflects an important GPU optimization principle: exploit on-chip memory whenever communication is local, and fall back to global memory only when the algorithm’s data dependencies require communication across blocks.

3 Results and Performance Analysis

3.1 Execution Time vs. Array Size

Figure 1 illustrates the execution time scaling across the three implementations for uniformly distributed data. The serial baseline exhibits expected $O(N \log N)$ growth. The OpenMP and CUDA implementations show significant reductions in overall compute time for arrays exceeding 1 million elements.

3.2 OpenMP Thread Scaling

Figure 2 demonstrates the speedup achieved by OpenMP relative to the serial baseline as the thread count increases.

As observed, speedup improves clearly from 1 to 2 threads, then exhibits diminishing and workload-dependent gains at higher thread counts (4, 8, and 16). This behavior is consistent with overheads from task management, synchronization, and memory-system limits, so scaling is not perfectly linear at high thread counts. Consequently, this scaling

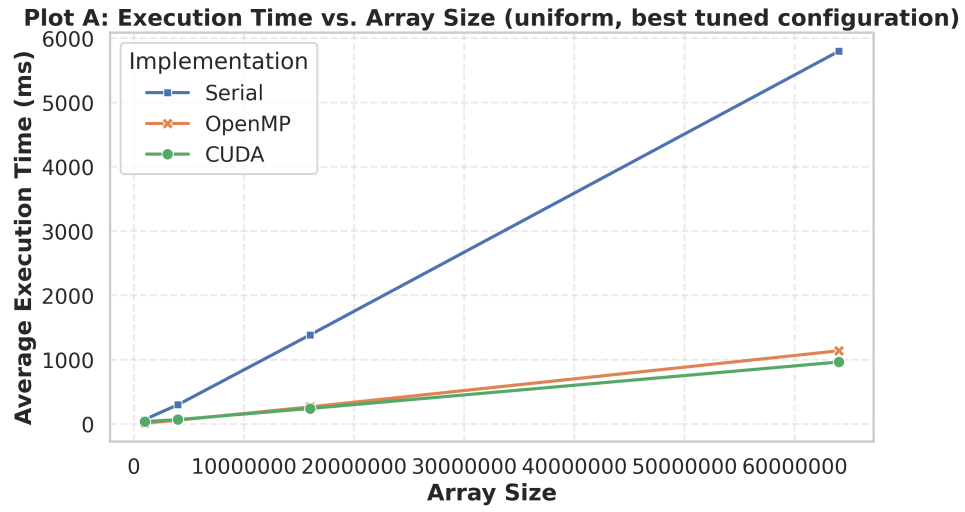


Figure 1: Execution Time vs. Array Size (Uniform Distribution). Compares Serial, OpenMP, and CUDA performance.

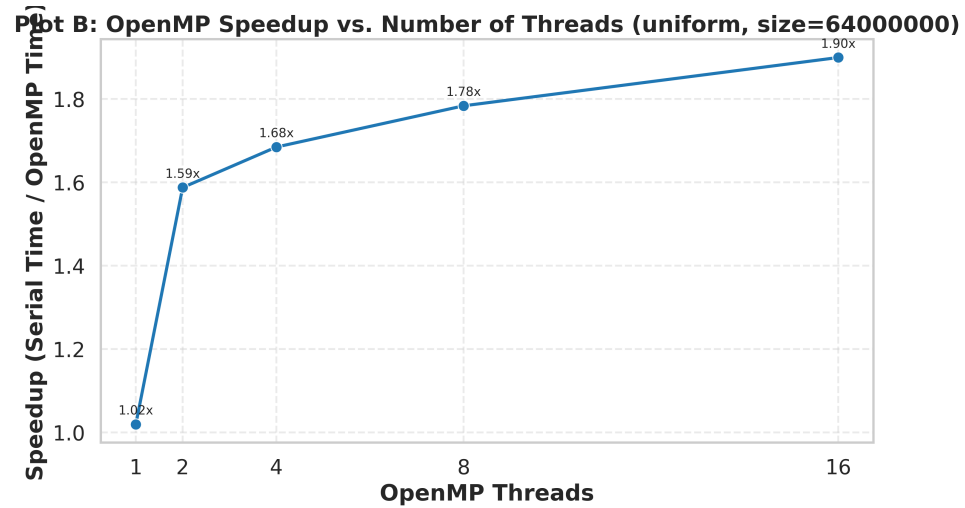


Figure 2: OpenMP Speedup vs. Number of Threads for $N = 64,000,000$.

behavior at high thread counts makes the OpenMP approach noticeably less efficient than GPU alternatives for massive datasets.

3.3 CUDA Block Size Optimization

Figure 3 analyzes the impact of varying CUDA block sizes (128, 256, 512).

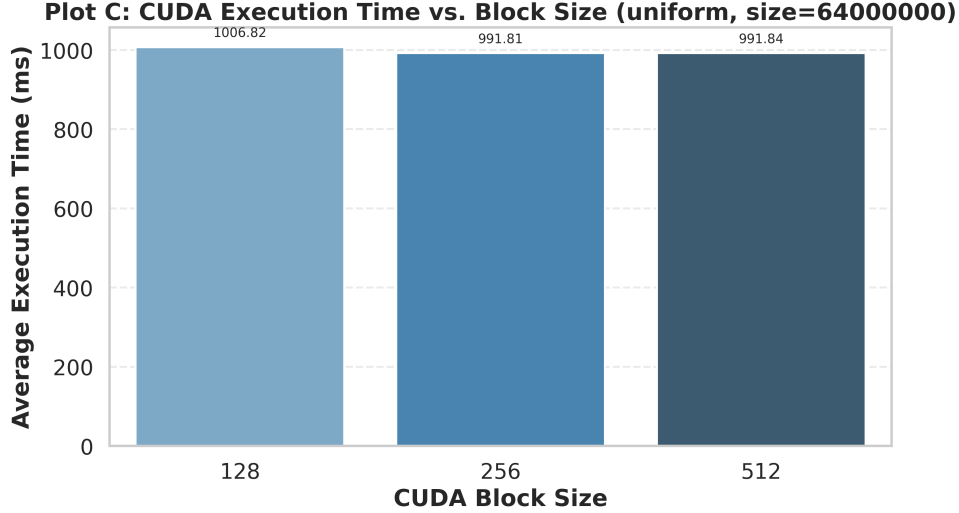


Figure 3: CUDA Execution Time across different Block Sizes.

The differences in execution time across block sizes highlight the importance of occupancy and warp scheduling. Empirically, block sizes of 256 and 512 were generally close and often outperformed 128 for the 64,000,000-element case, though the best choice varied slightly by distribution and run.

3.4 Algorithmic Scaling: Bitonic vs. MergeSort

A critical observation in the benchmark data is how the two algorithms scale differently depending on the input data distribution.

MergeSort’s performance is heavily data-dependent. As seen in the serial results, pre-sorted or reversed data requires drastically less time than uniformly random data. While the fundamental theoretical time complexity remains $O(N \log N)$, the structured data highly benefits modern CPU branch predictors during the merge step. However, scaling it across many CPU cores is hindered by the synchronization required to merge large adjacent sub-arrays.

Conversely, Bitonic Sort is a data-independent sorting network. It performs the exact same number of predetermined compare-and-swap operations regardless of whether the initial array is random or nearly sorted. While its theoretical time complexity $O(N \log^2 N)$ requires more total operations than MergeSort, it scales exceptionally well on GPUs. The SIMT (Single Instruction, Multiple Threads) architecture of the GPU can execute these non-branching operations in parallel across thousands of cores without thread divergence, allowing CUDA to massively outperform the CPU on large datasets despite the PCIe memory transfer penalties.

4 Conclusion

This project successfully validated the implementation of OpenMP and CUDA sorting algorithms against a serial baseline. The experiment matrix confirmed that while CPU parallelism (OpenMP) is effective for moderate workloads, it is ultimately constrained by serial merge bottlenecks. GPU acceleration (CUDA) utilizing sorting networks like Bitonic Sort provides superior scaling for massive datasets, provided the compute gains outweigh the host-to-device memory transfer overhead.